PROJECT OVERVIEW · INTERVIEW REFERENCE

🔬 AI Research Assistant

A Retrieval-Augmented Generation (RAG) chatbot that answers questions from uploaded PDF documents, with every answer traceable back to its exact source passage.

`Python` `Streamlit` `LangChain` `ChromaDB` `FastEmbed` `Groq LPU`

01 What it does

The user uploads one or more PDFs. The app reads the text, breaks it into small overlapping chunks, and converts each chunk into a numeric vector (embedding) that captures its meaning. These vectors are stored in a local vector database (ChromaDB). When the user asks a question, the app finds the most relevant chunks by meaning (not just keywords), sends them to a large language model hosted on Groq as context, and streams back an answer — along with the exact document and page number each part of the answer came from.

02 Project flow

Upload
User adds PDF(s) via Streamlit sidebar

→

Load
PyPDFLoader extracts text per page

→

Chunk
Text split into ~1000-char overlapping pieces

→

Embed
FastEmbed turns chunks into vectors

Store
Vectors saved in ChromaDB (local, on-disk)

→

Retrieve
Question → top-k most similar chunks

→

Generate
Groq LLM answers using retrieved context

→

Cite
Answer streamed back with source + page

03 Why these tools

- **Streamlit** — UI in pure Python, fast to build & deploy
- **LangChain** — glues loader/splitter/retriever/LLM together
- **ChromaDB** — local vector database for similarity search
- **FastEmbed** — lightweight ONNX embeddings, no GPU/torch needed
- **Groq** — runs the LLM on custom LPU chips for very fast, streamed responses

04 File-by-file breakdown

FILE	PURPOSE
root	
app.py	Main Streamlit app. Renders the UI, handles upload → processing pipeline, and the chat loop. This is the file you run.
requirements.txt	Exact, tested dependency versions so the install is reproducible with no version conflicts.
.env.example	Template for environment variables (Groq API key, model settings). Copied to `.env` locally.
core/	
config.py	Central settings: API key loading, default model, chunk size, top-k, file limits. Nothing here crashes on import if a key is missing.
prompts.py	The RAG system prompt template that instructs the model to answer only from the given context.
rag/ (the RAG pipeline)	
loader.py	Reads PDF files page-by-page using PyPDFLoader and tags each page with its source filename.
chunker.py	Splits loaded pages into smaller overlapping text chunks so retrieval can be precise.
embedder.py	Loads the FastEmbed embedding model that turns text chunks into vectors.
vector_store.py	Creates/loads the ChromaDB vector database and persists it to disk.
retriever.py	Wraps the vector store into a retriever that returns the top-k most relevant chunks for a query.
chatbot.py	Connects the retriever to the Groq-hosted LLM; builds the final prompt and streams the answer back, keeping track of which chunks were used (sources).
utils/ & assets/	
helpers.py	Small utilities: file-size formatting, exporting chat history to Markdown, API key sanity check.
style.css	Custom dark theme (ink-navy + amber accent) injected into the app for a non-default, branded look.
.streamlit/	
config.toml	Streamlit theme + server settings (upload limits, headless mode for deployment).

05 How to explain it in an interview

30-SECOND PITCH

"I built a RAG-based research assistant — you upload PDFs, and it lets you chat with them in natural language. Under the hood it chunks the documents, embeds them into a vector database, retrieves the most relevant pieces for each question, and passes them to an LLM hosted on Groq for a fast, streamed answer. Every answer shows exactly which document and page it came from, so it's grounded and verifiable, not just a hallucination."

06 Key talking points

- **Grounded answers:** the model only answers from retrieved context, and the prompt explicitly tells it to say so when the answer isn't in the documents — reduces hallucination.
- **Source citations:** every answer is paired with the source file and page number, shown in an expandable panel — builds user trust.
- **Lightweight embeddings:** chose FastEmbed (ONNX) over sentence-transformers/torch specifically so the app stays small enough to deploy free on Streamlit Cloud or Render.
- **Fast inference:** Groq's LPU hardware serves the LLM, so responses stream back noticeably faster than typical GPU-hosted APIs.
- **Config-driven:** model choice, temperature, chunk size, and retrieval depth are all adjustable at runtime from the sidebar — no code changes needed.

07 Likely follow-up questions

Q: What happens if the answer isn't in the PDF?

A: The prompt instructs the model to explicitly say it couldn't find the information, instead of guessing.

Q: How do you handle large documents?

A: Documents are split into overlapping chunks (default ~1000 characters) so only the most relevant pieces are sent to the model, keeping context small and answers focused.

Q: Why ChromaDB instead of another vector store?

A: It runs embedded/local with zero external setup, persists to disk, and integrates directly with LangChain — good fit for a self-contained, easily deployable app.

Q: How would you scale this further?

A: Swap ChromaDB for a managed vector DB (e.g. Pinecone/Qdrant Cloud), add OCR for scanned PDFs, add user auth, and cache embeddings so repeated documents aren't re-processed.